23CSE455 Design Patterns
Part 3 Findings: Cross-Language Comparison

Design Pattern Identification and Evolution in a Spring MVC-Based Warehouse Management Application

Team: B9
Date: 11 October 2026 (re-audited findings)
Team working repository: akdswordguy/Team_B9_DP
Companion documents: `report/main.tex` (full report), `report/part1_findings.tex`, `report/part2_findings.tex`

Roll number	Student name	Batch
AM.SC.U4CSE23215	D Aditya Kiran	CSE C
AM.SC.U4CSE23235	Mithun Krishna	CSE C
AM.SC.U4CSE23248	Sai Kishen K M	CSE C
AM.SC.U4CSE23359	Sivadev Smijith	CSE D

Evidence paths in this document are relative to `Aditya_findings/Part3-cross-language/` in the team working repository unless stated otherwise.

1 Selection and traceability

Part 3 implements the two GoF patterns confirmed in Part 1 in Java, Python, JavaScript and C++:

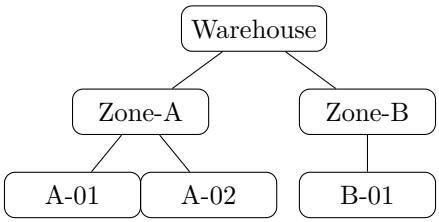
Pattern	OpenWMS source it is traced to (Part 1)	What the Part 3 versions do differently
Observer	`EventBroker`, `EventDispatcher`, `SimpleEventDispatcher`, `EventListener`, `RootApplicationEvent` at `9d263bf`	One `EventSubject` interface instead of the broker/dispatcher split; a single subscriber list rather than subscriptions keyed by event class; a concrete `RecordingObserver`, which OpenWMS CORE does not contain.
Composite	`TreeNode<T>`, `TreeNodeImpl<T>` at `9d263bf`	Separate `Container` and `Leaf` classes instead of one class with a leaf state, and a recursive `traverse` operation, which `TreeNode` lacks.

The eight implementations are independent reference implementations of the pattern concepts. They are not copies of the OpenWMS classes and do not modify OpenWMS. Each uses only its language's standard library.

2 Common specification

Observer. `subscribe`, `unsubscribe` and `publish` on a subject; an `Event` value with a string type; three recording observers. The scenario subscribes all three, publishes `STOCK_LOW`, unsubscribes observer 2, then publishes `STOCK_REPLENISHED`. The second event must reach observers 1 and 3 only, and event values must be preserved.

Composite. A shared component contract (`getName`, `addChild`, `removeChild`, `getChildren`, `traverse`) implemented by `Container` and `Leaf`. The hierarchy is:



Preorder traversal must return `Warehouse,Zone-A,A-01,A-02,Zone-B,B-01`. Leaves return no children and refuse additions.

3 Implementations and checks

3.1 Checks per language

The same checks exist in every language (names shown in Python/C++ form):

Pattern	Checks
Observer (3)	`test_all_observers_receive_first_event`; `test_unsubscribe_excludes_observer_from_second_event`; `test_event_values_and_object_are_preserved` (C++: `test_event_values_are_preserved`)
Composite (4)	`test_hierarchy_can_be_constructed`; `test_children_can_be_added_and_removed`; `test_leaf_rejects_children`; `test_recursive_traversal_order`

Java and C++ use executable assertion harnesses (`ObserverPatternTest`, `observer_test.cpp`, and so on), Python uses `unittest`, and JavaScript uses Node's built-in `node:assert/strict`. Reporting formats therefore differ, but the scenarios and assertions are equivalent.

3.2 Results

Pattern	Java	Python	JavaScript	C++
Observer	3/3	3/3	3/3	3/3
Composite	4/4	4/4	4/4	4/4

Total 28/28 checks. The results were obtained twice on different machines and toolchains:

Run	Toolchain	Result
Recorded run (Windows)	OpenJDK 25.0.4.1 (Temurin), Python 3.13.15, Node.js 24.21.0, g++ 16.2.0 (C++20)	28/28
Re-audit, 11 Oct 2026 (Linux)	OpenJDK 21.0.12.1, Python 3.13.16, Node.js 22.22.0, g++ 13.3.0 (`-std=c++20 -Wall -Wextra -Werror -pedantic`)	28/28

Per-harness console logs were not retained in the repository; the exact commands are in `PART3_EVIDENCE.md`.

4 Static metrics

LOC is physical lines including blanks and comments. Maximum cyclomatic complexity (CCN) is from Lizard 1.24.1 over both files in each directory. Every value below was recomputed in the re-audit and matches `metrics/cross_language.csv`.

Pattern	Language	Impl.	Test	Total	Max CCN	Function(s) at max
Observer	Java	65	70	135	2	`Publisher.publish`, `Publisher.subscribe`, `check`
Observer	Python	52	37	89	3	`setUp`, `test_event_values_and_object_are_preserved`
Observer	JavaScript	54	43	97	2	`Publisher.subscribe`, `Publisher.publish`
Observer	C++	55	61	116	2	`Publisher::publish`, `Publisher::subscribe`, one test
Composite	Java	89	63	152	2	`Container.traverse`, `check`, one test
Composite	Python	71	48	119	2	`Container.remove_child`, `Container.traverse`
Composite	JavaScript	81	47	128	2	`Container.removeChild`
Composite	C++	85	62	147	2	`Container::removeChild`, `Container::traverse`, one test

Structure (`metrics/pattern_metrics.csv`). Every Observer version has 5 named types (3 concrete including the `Event` value, 2 abstract contracts), inheritance depth 1 and 3 direct dependency edges; the C++ `Event` is a struct. Every Composite version has 3 named types (2 concrete, 1 abstract), inheritance depth 1 and one dependency edge (container to component). Algorithmically, subscribe/unsubscribe and publish are linear in the number of subscribers, traversal is $O(V + E)$, and child add/remove are linear in the number of children.

5 Dynamic metrics

Each harness was run as a separate process: one untimed warm-up and 30 timed runs, measured from process launch to exit with the same PowerShell stopwatch; Java and C++ compilation was excluded. Values are the recorded medians from `metrics/cross_language.csv` and were not re-measured in the re-audit.

Pattern	Language	Median process time (ms)	Runs
Observer	Java	175.1	30
Observer	Python	279.3	30
Observer	JavaScript	128.6	30
Observer	C++	33.1	30
Composite	Java	154.0	30
Composite	Python	264.6	30
Composite	JavaScript	130.2	30
Composite	C++	33.4	30

These programs do microseconds of real work, so the times are almost entirely runtime start-up (JVM, Python interpreter, Node, or a native executable). They show start-up cost, not the cost of

the patterns, and do not support a language-performance ranking.

6 Comparison

6.1 How each language expresses the contracts

Language	Contract mechanism	Observer storage	Composite children
Java	`interface`	`ArrayList<Observer>`	`List<Component>`, immutable snapshot from `getChildren`
Python	`abc.ABC` with abstract methods	`list`, iterated as a `tuple` copy	`list`, returned as a `tuple` snapshot
JavaScript	Base class whose methods throw until overridden	private `#observers` array	array, shallow copy from `getChildren`
C++	Pure virtual class	`std::vector<Observer*>` (non-owning)	`std::vector<std::shared_ptr<Component>>`

6.2 Observer

The subject depends only on the observer contract, so new observers can be added without changing it. All four versions ignore duplicate subscriptions and notify from a copy of the subscriber list, so an observer can unsubscribe during notification safely, as in Python:

```
def publish(self, event: Event) -> None:
    for observer in tuple(self._observers):
        observer.update(event)
```

OpenWMS's `SimpleEventDispatcher` solves the same problem with a concurrent set per event class, and it also supports subscription by Spring bean name. The C++ version stores raw pointers, so a subscribed observer must outlive its registration; the other three use managed references.

6.3 Composite

Clients call `getName`, child operations, `getChildren` and `traverse` without checking the concrete type. `Container.traverse` returns its own name and then recurses into each child, which gives preorder. Separating `Leaf` from `Container` lets a leaf reject `addChild` explicitly; OpenWMS's single `TreeNodeImpl` instead lets any node gain children.

6.4 Language observations

1. **Type system.** Java and C++ check contracts at compile time; Python ABCs enforce abstract methods at instantiation; JavaScript relies on convention and runtime errors.
2. **Boilerplate.** Python is shortest. The C++ and Java totals are inflated by explicit types and harness code, and LOC is not normalized for formatting.
3. **Ownership.** Only C++ makes ownership explicit: `shared_ptr` for Composite children and non-owning pointers for observers.
4. **Complexity.** Maximum CCN is 2 to 3 everywhere. That reflects how small the examples are, not design quality.

7 Limitations

- These are teaching-sized reference implementations, not production event or location subsystems, and they do not modify OpenWMS.
- LOC is physical lines; CCN depends on Lizard 1.24.1's parsers for each language.
- Timings are dominated by process start-up and were measured once on one machine.
- Raw console logs from the recorded run were not retained.

8 Re-audit summary

No incorrect values were found in Part 3. All eight harnesses passed again on a second toolchain, and all sixteen file LOC values and eight CCN values were recomputed and match. The only additions are the re-run record in `PART3_EVIDENCE.md`, the AUDIT-01 entry in `metrics/llm_records.csv` and `LLM_RECORDS.md`, and the comparison with the OpenWMS originals in Sections 1 and 6.

9 Evidence files

- `observer/\{java,python,javascript,cpp\}/` and `composite/\{java,python,javascript,cpp\}/` — implementation and test source.
- `PART3_EVIDENCE.md` — specification, commands, results, metrics, comparison, re-run record.
- `metrics/cross_language.csv` (8 rows, 19 columns), `metrics/pattern_metrics.csv`, `metrics/llm_records.csv`.
- `Aditya_findings/LLM_RECORDS/LLM_RECORDS.md` — records P3-01, P3-02 and AUDIT-01.